

Comunicaciones en red



Caso práctico

María y Juan han iniciado un nuevo proyecto que le han encargado a la empresa BK, en el que le piden que realicen un programa para que muchos clientes puedan añadir información y ésta se centralice para ser consultada por todos.



Q [Ministerio de Educación](#). (Uso educativo [nc](#))

María: —Hola **Juan**, nos han encargado un proyecto muy interesante en el que tenemos que realizar una aplicación utilizando sockets.

Juan: —¿Sockets? He oído hablar de ellos, pero todavía nos lo he utilizado.

María: —Los sockets permiten que las aplicaciones puedan comunicarse por red para transmitir datos. Por ejemplo, el servicio más importante de Internet, las páginas web, utiliza sockets. No te preocupes, vamos a realizar la aplicación, pero antes repasaremos los conceptos más importantes de redes.

El objetivo de la unidad es que aprendas a crear aplicaciones que utilicen sockets TCP y/o UDP. Para ello, primero se realiza un breve repaso sobre los conceptos básicos de redes haciendo un especial hincapié sobre la capa de transporte que define las conexiones TCP y UDP.

Una vez visto los conceptos básicos de redes, aprenderás a crear sockets TCP en java siguiendo el modelo cliente/servidor. Finalmente, aprenderá a utilizar los sockets UDP en java.



Q  [Ministerio de Educación, Formación Profesional y Deportes \(Dominio público\)](#)

Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

 [Aviso Legal](#) 

1.- Conceptos básicos



Caso práctico

Juan ha empezado a realizar la programación.

María: —Mira **Juan**, las aplicaciones que se comunican con Sockets lo realizan a través de una determinada dirección IP y un determinado puerto.

Básicamente, las direcciones IP definen a qué ordenador van los datos y el puerto indica, dentro del ordenador, a qué programa van los datos.

Juan: —Ya me suena.... por ejemplo, la dirección IP indica el servidor web a donde van los datos y el puerto indicaría que es un servicio web, FTP,..

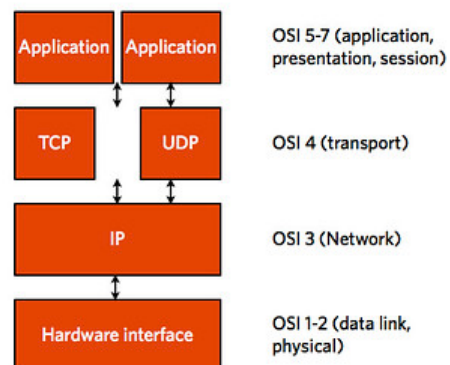
María: —Sí exacto. Veo que esto lo tienes más o menos claro pero vamos a profundizar un poco más. Ahora lo importante es ver los tipos de conexión que existen (TCP o UDP) porque tenemos que seleccionar un tipo.



Q [Ministerio de Educación](#). (Uso educativo [nc](#))

Internet es a día de hoy una herramienta esencial en la operativa cotidiana de casi cualquier empresa (independientemente de su actividad) y, obviamente, la utilización de redes es un activo básico. De hecho es totalmente necesario disponer de una red interna que le permita compartir información y recursos (por ejemplo impresoras en red). Esta situación también se ha extendido, desde hace ya años, a nuestros hogares.

TCP /IP stack



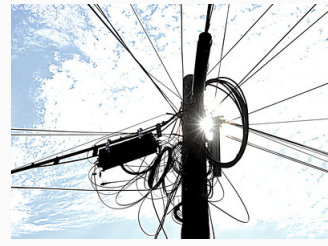
Q [Purple Slog](#) (CC BY-SA)

Multitud de empresas, de diversos tamaños, hacen uso de aplicaciones, que se comunican a través de Internet, para poder compartir información entre sus empleados. Por ejemplo: aplicaciones de gestión y facturación que permiten que varias tiendas puedan realizar de forma centralizada la facturación de toda la empresa, gestionar el stock, herramientas para imputación de tareas y jornada laboral, etc.

A continuación vamos a ver los conceptos más importantes sobre redes, necesarios para más adelante poder programar nuestras aplicaciones. Para ello, en primer lugar, veremos una pequeña introducción sobre el modelo TCP/IP, los tipos de conexiones que se pueden realizar, así como los modelos más importantes de comunicaciones.

1.1.- Familia de protocolos de Internet

En 1969 la agencia ARPA (*Advanced Research Projects Agency*) del Departamento de Defensa de los Estados Unidos inició un proyecto de interconexión de ordenadores mediante redes telefónicas. Al ser un proyecto desarrollado por militares en plena guerra fría un principio básico de diseño era que la red debía poder resistir la destrucción de parte de su infraestructura (por ejemplo a causa de un ataque nuclear), de forma que dos nodos cualesquiera pudieran seguir comunicados siempre que hubiera alguna ruta que los uniera. Esto se consiguió en 1972 creando una red de conmutación de paquetes denominada ARPAnet, la primera de este tipo que operó en el mundo. La conmutación de paquetes unida al uso de topologías malladas mediante múltiples líneas punto a punto dio como resultado una red altamente fiable y robusta.



Q Mike Bitzenhofer (CC BY-SA)

ARPAnet

ARPAnet fue creciendo paulatinamente, y pronto se hicieron experimentos utilizando otros medios de transmisión de datos, en particular enlaces por radio y vía satélite; los protocolos existentes tuvieron problemas para interoperar con estas redes, por lo que se diseñó un nuevo conjunto o pila de protocolos, y con ellos una arquitectura. Este nuevo conjunto se denominó TCP/IP (Transmission Control Protocol/Internet Protocol), nombre que provenía de los dos protocolos más importantes que componían la pila; la nueva arquitectura se llamó sencillamente modelo TCP/IP. A la nueva red, que se creó como consecuencia de la fusión de ARPAnet con las redes basadas en otras tecnologías de transmisión, se la denominó Internet.

OSI y TCP/IP

La aproximación adoptada por los diseñadores del TCP/IP fue mucho más pragmática que la de los autores del modelo OSI. Mientras que en el caso de OSI se emplearon varios años en definir con sumo cuidado una arquitectura de capas donde la función y servicios de cada una estaban perfectamente definidas, y sólo después se planteó desarrollar los protocolos para cada una de ellas, en el caso de TCP/IP la operación fue a la inversa; primero se especificaron los protocolos, y luego se definió el modelo como una simple descripción de los protocolos ya existentes. Por este motivo el modelo TCP/IP es mucho más simple que el OSI. También por este motivo el modelo OSI se utiliza a menudo para describir otras arquitecturas, como por ejemplo TCP/IP, mientras que el modelo TCP/IP nunca suele emplearse para describir otras arquitecturas.

que no sean la suya propia.

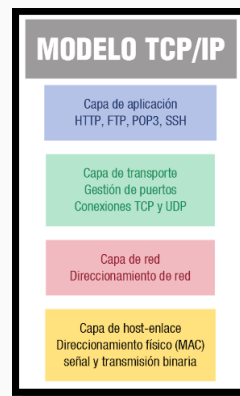
La implementación TCP/IP tiene sólo cuatro capas (en contraposición al modelo OSI que tiene 7 capas):

Capa de host-red

La capa host-red. Esta capa permite comunicar el ordenador con el medio que conecta el equipo a la red. Para ello primero debe permitir convertir la información en impulsos físicos (p.ej. eléctricos, magnéticos, luminosos) y además, debe permitir las conexiones entre los ordenadores de la red. En esta capa se realiza un direccionamiento físico utilizando las direcciones MAC.

Capa de red

La capa de red. Esta capa es el eje de la arquitectura TCP/IP ya que permite que los equipos envíen paquetes en cualquier red y viajen de forma independiente a su destino (que podría estar en una red diferente). Los paquetes pueden llegar incluso en un orden diferente a aquel en que se enviaron, en cuyo caso corresponde a las capas superiores reordenándolos, si se desea la entrega ordenada. La capa de red define un formato de paquete y protocolo oficial llamado IP (Internet Protocol). El trabajo de la capa de red es entregar paquetes IP a su destino. Aquí la consideración más importante es decidir el camino que tienen que seguir los paquetes (encaminamiento), y también evitar la congestión. En esta capa se realiza el direccionamiento lógico o direccionamiento por IP, ya que ésta es la capa encargada de enviar un



Q Ministerio de Educación. (Uso educativo nc)

determinado mensaje a su dirección IP de destino.

Capa de transporte

La capa de transporte. La capa de transporte permite que los equipos lleven a cabo una conversación. Aquí se definieron dos protocolos de transporte: TCP (Transmission Control Protocol) y UDP (User Datagram Protocol). El protocolo TCP es un protocolo orientado a conexión y fiable, y el protocolo UDP es un protocolo no orientado a conexión y no fiable. En esta capa además se realiza el direccionamiento por puertos. Gracias a la capa anterior, los paquetes viajan de un equipo origen a un equipo destino. La capa de transporte se encarga de que la información se envíe a la aplicación adecuada (mediante un determinado puerto).

Capa de aplicación

La capa de aplicación. Esta capa engloba las funcionalidades de las capas de sesión, presentación y aplicación del modelo OSI. Incluye todos los protocolos de alto nivel relacionados con las aplicaciones que se utilizan en Internet (por ejemplo HTTP, FTP, TELNET).



Debes conocer

Si quieres ampliar tus conocimientos sobre el modelo TCP/IP debes leer el siguiente [texto del enlace: Modelo TCP/IP](#)  prestando especial atención a la capa de transporte.



Autoevaluación

Indica la capa que se encarga de enrutar un paquete por la red para que llegue a su destino.

- ☐ Capa host-red.
- ☐ Capa de red.
- ☐ Capa de transporte.
- ☐ Capa de aplicación.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-7_58)
2. Correcto (#answer-7_82)
3. Incorrecto (#answer-7_84)
4. Incorrecto (#answer-7_86)

1.2.- Protocolos TCP/UDP

Como se ha visto anteriormente, la capa de transporte cumple la función de establecer las reglas necesarias para establecer una conexión entre dos dispositivos.

Desde la capa anterior, la capa de red, la información se recibe en forma de paquetes desordenados y la capa de transporte debe ser capaz de manejar dichos paquetes y

obtener un único flujo de datos. Recuerde, que la capa de red en la arquitectura TCP/IP no se preocupa del orden de los paquetes ni de los errores, es en esta capa donde se deben cuidar estos detalles.



Q Kev (CC BY-SA)

La capa de transporte del modelo TCP/IP es equivalente a la capa de transporte del modelo OSI, por lo que es el encargado de la transferencia libre de errores de los datos entre el emisor y el receptor, aunque no estén directamente conectados, así como de mantener el flujo de la red. La tarea de este nivel es proporcionar un transporte de datos confiable de la máquina de origen a la máquina destino, independientemente de la red física.

Existen dos tipos de protocolos:

- **TCP (Transmission Control Protocol).** Es un protocolo orientado a la conexión que permite que un flujo de bytes originado en una máquina se entregue sin errores en cualquier máquina destino. Este protocolo fragmenta el flujo entrante de bytes en mensajes y pasa cada uno a la capa de red. En el diseño, el proceso TCP receptor reensambla los mensajes recibidos para formar el flujo de salida. TCP también se encarga del control de flujo para asegurar que un emisor rápido no pueda saturar a un receptor lento con más mensajes de los que pueda gestionar.
- **UDP (User Datagram Protocol).** Es un protocolo sin conexión, para aplicaciones que no necesitan la asignación de secuencia ni el control de flujo TCP y que desean utilizar los suyos propios. Este protocolo también se utilizan para las consultas de petición y respuesta del tipo cliente-servidor, y en aplicaciones en las que la velocidad es más importante que la entrega precisa, como las transmisiones de voz o de vídeo. Uno de sus usos es en la transmisión de audio y vídeo en tiempo real, donde no es posible realizar retransmisiones por los estrictos requisitos de retardo que se tiene en estos casos.

De esta forma, a la hora de programar nuestra aplicación deberemos elegir el protocolo que queremos utilizar según nuestras necesidades: TCP o UDP.



Para saber más

Es interesante que leas algo más sobre los protocolos más importantes de este nivel, por lo que te proponemos los siguientes enlaces:

Protocolo TCP 

Protocolo UDP 



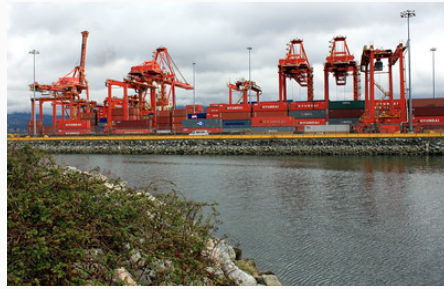
Autoevaluación

Las conexiones garantizan la correcta recepción de los paquetes y las conexiones son más rápidas pero no permiten la corrección de errores.

Enviar

1.3.- Puertos de comunicación

Con la capa de red se consigue que la información vaya de un equipo origen a un equipo destino a través de su dirección IP. Pero para que una aplicación pueda comunicarse con otra aplicación es necesario establecer a qué aplicación se conectará. El método que se emplea es el de definir direcciones de transporte en las que los procesos pueden estar a la escucha de solicitudes de conexión. Estos puntos terminales se llaman puertos.



Q [Chris Huggins](#) (CC BY-SA)

Aunque muchos de los puertos se asignan de manera arbitraria, ciertos puertos se asignan, por convenio, a ciertas aplicaciones particulares o servicios de carácter universal. De hecho, la [IANA](#) (Internet Assigned Numbers Authority) determina las asignaciones de todos los puertos. Existen tres rangos de puertos establecidos:

- **Puertos conocidos [0, 1023].** Están reservados para aplicaciones de uso estándar. Entre ellos cabe destacar por su popularidad
 - 21: FTP (File Transfer Protocol)
 - 22: [SSH](#) (Secure Shell)
 - 23: Telnet
 - 25: SMTP (Simple Mail Transfer Protocol)
 - 53: [DNS](#) (Servicio de nombres de dominio)
 - 80: HTTP (Hypertext Transfer Protocol)
 - 443: HTTPS/SSL (Hypertext Transfer Protocol Secure/Secure Sockets Layer)
- **Puertos registrados [1024, 49151].** Son asignados por IANA para un servicio específico o aplicaciones. Estos puertos pueden ser utilizados por los usuarios libremente.
- **Puertos dinámicos [49152, 65535].** Este rango de puertos no puede ser registrado y su uso se establece para conexiones temporales entre aplicaciones.

Cuando se desarrolla una aplicación que utilice un puerto de comunicación, optaremos por utilizar puertos comprendidos entre el rango 1024-49151.



Para saber más

Durante el desarrollo de este módulo y de otros de este ciclo, necesitaras conocer cuales son los puertos relacionados con cada una de las aplicaciones, por tanto te recomendamos el siguiente enlace:

[Puertos de comunicaciones](#)



Autoevaluación

¿Qué puerto se utiliza para el servicio web?

- ☐ 21
- ☐ 80
- ☐ 8080
- ☐ 22

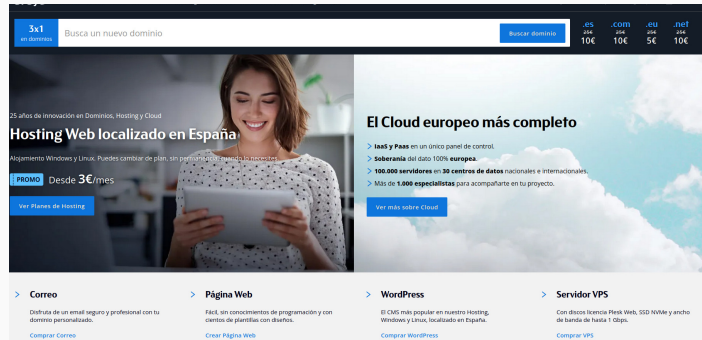
Mostrar retroalimentación

Solución

1. Incorrecto (#answer-15_58)
2. Correcto (#answer-15_91)
3. Incorrecto (#answer-15_93)
4. Incorrecto (#answer-15_95)

1.4.- Nombres en Internet

Los equipos informáticos se comunican entre sí mediante direcciones IP (por ejemplo 193.147.0.29). Sin embargo nosotros preferimos utilizar nombres como



Q Arsysis (CC BY)

www.educacionfpydeportes.gob.es  porque son más fáciles de recordar y porque ofrecen la flexibilidad de poder cambiar la máquina en la que están alojados (cambiaría entonces la dirección IP) sin necesidad de cambiar las referencias a él.

El sistema de resolución de nombres (DNS) basado en dominios, en el que se dispone de uno o más servidores encargados de resolver los nombres de los equipos pertenecientes a su ámbito, logra, por un lado, la centralización necesaria para la correcta sincronización de los equipos empleando un sistema jerárquico que permite una administración focalizada y, por otro lado, una gestión descentralizada y un mecanismo de resolución eficiente.

A la hora de realizar la comunicación con un equipo es posible hacerlo directamente a través de su dirección IP o indicando su entrada DNS (p.ej. *servidor.miempresa.com*). En el caso de utilizar la entrada DNS el equipo resuelve automáticamente su dirección IP a través del servidor de nombres que utilice en su conexión a Internet.



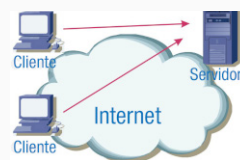
Para saber más

DNS no fue diseñado con la mente puesta en la seguridad, y hay muchos tipos de ataques creados para explotar las vulnerabilidades en el sistema DNS. En el siguiente enlace, ubicado en el sitio web de uno de los principales proveedores de servicio DNS a nivel mundial, podrás explorar más información acerca de ello:

[Ataques DNS: ¿cómo funcionan?](#) 

1.5.- Modelos de comunicaciones

Sin duda alguna, el modelo de comunicación que ha revolucionado los sistemas informáticos es el modelo cliente/servidor. El modelo cliente/servidor esta compuesto por un servidor que ofrece una serie de servicios y unos clientes que acceden a dichos servicios a través de la red.



Q Ministerio de Educación. (Uso educativo nc)

Por ejemplo, el servicio más importante de Internet, el WWW , utiliza el modelo cliente/servidor. Por un lado tenemos el servidor que alojan las páginas web y por otro lado los clientes que solicitan al servidor una determinada página web.



Q Ministerio de Educación. (Uso educativo nc)

Otro modelo ampliamente utilizado son los Sistemas de Información Distribuidos. Un Sistema de Información Distribuido esta compuesto por un conjunto de equipos que interactúan entre sí y pueden trabajar a la vez como cliente y servidor. Desde el punto de

vista externo es igual que un sistema cliente/servidor ya que el cliente ve al Sistema de Información Distribuido como una entidad. Internamente, los equipos del Sistema de Información Distribuido interactúan entre sí (actuando como servidores y clientes de forma simultánea) para compartir información, recursos, realizar tareas, etc.



Autoevaluación

Rellena los huecos con las palabras que faltan.

El suele tener un servidor mientras que esta compuesto por muchos servidores.

Enviar

2.- Sockets TCP



Caso práctico

Juan va a hablar con **María** para debatir lo aprendido.

Juan: —Hola **María**, ya lo he entendido. Tenemos dos tipos de conexiones: TCP que son orientadas a conexión y se utilizan para enviar datos de aplicaciones, y UDP que son no orientadas a conexión y se utilizan cuando queremos enviar datos muy rápido pero no nos importa que se pierda algún paquete.

María: —Exacto Juan. Para ponerte un ejemplo, las conexiones TCP se utilizan para enviar ficheros (p.ej. el servicio FTP) y las comunicaciones UDP se utilizan para enviar música y vídeo en tiempo real. Como tenemos claro que la aplicación que vamos a realizar va a utilizar conexiones TCP, ya que no queremos que se pierda ningún dato, a continuación vamos a aprender a programar con java los sockets TCP.



Q WOCinTech Chat (<https://www.flickr.com/photos/wocintechchat/25392390163/>). (CC BY)

Los sockets permiten la comunicación entre procesos de diferentes equipos de una red. Un socket es un punto de información por el cual un proceso puede recibir o enviar información.

Tal y como hemos visto anteriormente, existen dos tipos de protocolos de comunicación: TCP o UDP. En el protocolo TCP es un protocolo orientado a la conexión que permite que un flujo de bytes originado en una máquina se entregue sin errores en cualquier máquina destino. Por otro lado, el protocolo UDP, no orientado a conexión, se utiliza para comunicaciones donde se prioriza la velocidad sobre la pérdida de paquetes.

A la hora de crear un socket hay que tener claro el tipo de socket que se quiere crear (TCP o UDP). En esta sección vamos a aprender a utilizar los sockets TCP y en el siguiente punto veremos los sockets UDP.

Al utilizar sockets TCP, el servidor utiliza un puerto por el que recibe las diferentes peticiones de los clientes. Normalmente, el puerto del servidor es un puerto bajo [1-1023].



Q Ministerio de Educación. (Uso educativo nc)

Cuando el cliente realiza la conexión con el servidor, a partir de ese momento se crea un nuevo socket que será el encargado de permitir el envío y recepción de datos entre el cliente/servidor. El puerto se crea de forma dinámica y se

encuentra en el rango 49152 y 65535. De ésta forma, el puerto por donde se reciben las conexiones de los clientes queda libre y cada comunicación tiene su propio socket.



Q Ministerio de Educación. (Uso educativo no)

El paquete `java.net` de Java proporciona la clase `Socket` que permite la comunicación por red. De forma adicional, `java.net` incluye la clase `ServerSocket`, que permite a un servidor escuchar y recibir las peticiones de los clientes por la red. Y la clase `Socket` permite a un cliente conectarse a un servidor para enviar y recibir información.



Para saber más

A continuación se estudian las llamadas más importantes de [sockets en Java](#). Si deseas complementar dicha información, en el siguiente enlace puedes ver la documentación oficial de Sockets en java.



Autoevaluación

Indica si la siguiente información es correcta. Cuando te conectas al puerto 80 TCP, ¿todas las comunicaciones van por ese puerto?

☐ Verdadero ☐ Falso

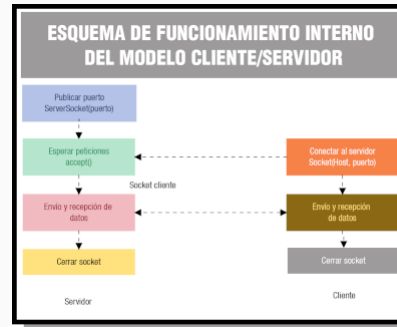
Falso

Es falso ya que la petición se realiza por el puerto 80 pero la transferencia de datos se realiza por un puerto superior.

2.1.- Servidor

Los pasos que realiza el servidor para realizar una comunicación son los siguientes:

- **Publicar puerto:** se emplea el constructor de la clase `ServerSocket` indicando el puerto por donde se recibirán las conexiones.
- **Esperar peticiones:** en este momento el servidor queda a la espera a que se conecte un cliente. Una vez que se conecte un cliente se crea el `socket` del cliente por donde se envían y reciben los datos.
- **Envío y recepción de datos:** para recibir/enviar datos es necesario crear un flujo (*stream*) de entrada y otro de salida. Cuando el servidor recibe una petición, éste la procesa y le envía el resultado al cliente.
- Una vez finalizada la comunicación se **cierra el socket del cliente**.



Q Ministerio de Educación. (Uso educativo nc)

Para publicar el puerto del servidor se utiliza la función `ServerSocket` a la que hay que indicarle el puerto a utilizar. Su estructura es:

```
ServerSocket socketServidor = new ServerSocket(puerto);
```

Una vez publicado el puerto, el servidor utiliza la función `accept()` para esperar la conexión de un cliente. Una vez que el cliente se conecta, entonces se crea un nuevo socket por donde se van a realizar todas las comunicaciones con el cliente y servidor.

```
Socket socketCliente = socketServidor.accept();
```

Una vez recibida la petición del cliente el servidor se comunica con el cliente a través de streams de datos que veremos en el siguiente punto.

Finalmente, una vez terminada la comunicación se cierra el socket de la siguiente forma:

```
socketCliente.close();
```

Ejemplo

A continuación se muestra un extracto del código (con comentarios explicativos) de un programa que ejecuta el rol de servidor:

```
...
```

```
try {
```

```
    ServerSocket socketServidor = new ServerSocket(puerto); // se inicia
                                                                // la escucha del servidor en el puerto indicado
```



```

// mensaje por salida estándar que informa el puerto en el que se
// realiza la escucha
System.out.println("Inicio de escucha en el puerto " + puerto);

Socket socketCliente = socketServidor.accept(); // espera activa hasta
// que se conecte un cliente. En ese momento se realiza la
// creación de un socket para dicho cliente
// Hasta que no se conecte un cliente el puerto permanece
// ocupado y el proceso permanece en estado de ejecución

/**
 * -----
 * en este bloque se establece el código (lógica de negocio) que es
 * necesario para atender la petición recibida por el cliente
 * -----
 */

socketCliente.close(); // cuando se ha finalizado de atender al
// cliente se procede a cerrar el socket y
// liberar los recursos asociados a éste
} catch (IOException e) {
    System.out.println(e.getMessage()); // se imprime por la salida
// estándar el mensaje de error
}

...

```



Código fuente del Servidor

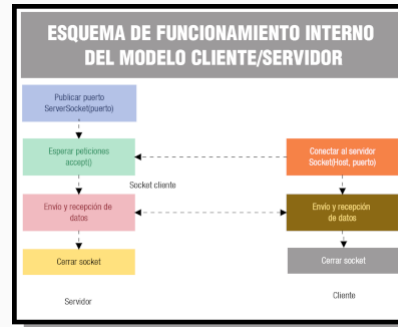
Aquí puedes descargar el código fuente completo:

[📄 Creación de Servidor empleando sockets](#) 

2.2.- Cliente

Los pasos que realiza el cliente para realizar una comunicación son:

- **Conectarse con el servidor.** El cliente utiliza la función Socket para indicarse con un determinado servidor a un puerto específico. Una vez realizada la conexión se crea el socket por donde se realizará la comunicación.



Q Ministerio de Educación. (Uso educativo nc)

- **Envío y recepción de datos.** Para poder recibir/enviar datos es necesario crear un flujo (stream) de entrada y otro de salida.
- Una vez finalizada la comunicación se **cierra el socket**.

Para conectarse a un servidor se utiliza el método constructor `Socket` indicando el equipo y el puerto al que desea conectarse. Su sintaxis es:

```
Socket socketCliente = new Socket(host, puerto);
```

donde *host* es un `String` que guarda el nombre o dirección IP del servidor y *puerto* es una variable del tipo `int` que almacena el valor del puerto.

Si lo prefiere también puede realizar la conexión directamente:

```
Socket socketCliente = new Socket("192.168.1.200", 1500);
```

Una vez establecida la comunicación, se crean los streams de entrada y salida para realizar las diferentes comunicaciones entre el cliente y el servidor. En el siguiente apartado veremos la creación de streams.

Finalmente, una vez terminada la comunicación se cierra el socket de la siguiente forma:

```
socketCliente.close();
```

Ejemplo

A continuación se muestra un extracto del código (con comentarios explicativos) de un programa que ejecuta el rol de cliente:

```
...
```

```
try {
    Socket socketCliente = new Socket(host, puerto); // primitiva que
    // permite al cliente la conexión, en el host y puerto indicados,
    // con el servidor. Si no hubiera ningún proceso atendiendo
    // en esa máquina/puerto la petición se dispara una
    // excepción (indicada más abajo).
```

```

/**
 * -----
 * en este bloque se establece el código (lógica de negocio) que es
 * necesario para realizar la petición enviada por el cliente
 * -----
 */

socketCliente.close(); // cuando se ha finalizado de realizar la
                        // solicitud del cliente se procede a cerrar
                        // el socket y liberar los recursos asociado
                        // a éste
} catch (IOException e) {
    System.out.println(e.getMessage()); // se imprime por la salida
                                        // estándar el mensaje de error
}

...

```



Código fuente del Cliente

Aquí puedes descargar el código fuente completo:

 [Creación de Cliente empleando sockets.](#) 



Autoevaluación

¿Qué función permite al servidor publicar un puerto?

- ☐ Socket.
- ☐ ServerSocket.
- ☐ Accept.
- ☐ ExportSocket.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-24_58)

2. Correcto (#answer-24_112)
3. Incorrecto (#answer-24_114)
4. Incorrecto (#answer-24_116)

2.3.- Flujo de Entrada y de Salida

Una vez establecida la conexión entre el cliente y el servidor se inicializa la variable de la clase `Socket` que en el ejemplo anterior se llamaba `socketCliente`. Para poder enviar o recibir datos a través del `socket` es necesario establecer un *stream* (flujo) de entrada o de salida según corresponda.



Q Ministerio de Educación. (Uso educativo [nc](#))

Continuando con el ejemplo anterior se muestra cómo establecer un *stream* de salida llamado `flujo_salida`:

```
OutputStream aux = socketCliente.getOutputStream();  
DataOutputStream flujo_salida= new DataOutputStream(aux);
```

o lo que es lo mismo,

```
DataOutputStream flujo_salida= new DataOutputStream(socketCliente.getOutputStream());
```

A partir de éste momento puede enviar información de la siguiente forma:

```
flujo_salida.writeUTF("Enviar datos...");
```

De forma análoga, puede establecer el *stream* de entrada de la siguiente forma:

```
InputStream aux = socketCliente.getInputStream();  
DataInputStream flujo_entrada = new DataInputStream( aux );
```

o lo que es lo mismo,

```
DataInputStream flujo_entrada = new DataInputStream(socketCliente.getInputStream());
```

A continuación se muestra una forma cómoda de recibir información:

```
String datos = flujo_entrada.readUTF();
```



Para saber más

Además de utilizar las funciones `writeUTF` y `readUTF` es posible recibir información de otras formas. Para profundizar puede consultar los

siguientes enlaces:

 [Data Input Stream](#) 

 [Data Output Stream](#) 



Autoevaluación

Indica si la siguiente afirmación es correcta. ¿Existen flujos de entrada y salida simultáneos?

☐ Verdadero.

☐ Falso.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-28_58)

2. Correcto (#answer-28_119)

2.4.- Ejemplo

Para continuar con el ejemplo anterior y poder utilizar los `sockets` para enviar información vamos a realizar un ejemplo muy sencillo en el que el servidor va a aceptar tres clientes (de forma secuencial no concurrente) y le va a indicar el número de cliente que es.

Implementación del servidor

```
...
ServerSocket socketServidor = new ServerSocket(puerto); // se inicia
// la escucha del servidor en el puerto indicado

// mensaje por salida estándar que informa el puerto en el que se
// realiza la escucha
System.out.println("Inicio de escucha en el puerto " + puerto);
for (int numeroClientes = 1; numeroClientes <= 3; numeroClientes++) {
    // espera activa hasta que se conecte un cliente. En ese
    // momento se realiza la creación de un socket para dicho
    // cliente. Hasta que no se conecte un cliente el puerto
    // permanece ocupado y el proceso permanece en estado de
    try (Socket socketCliente = socketServidor.accept()) { // ejecución
        System.out.println("Se está atendiendo al cliente '" + numeroClientes
        DataOutputStream flujo_salida= new DataOutputStream(socketCliente.get
        flujo_salida.writeUTF("Hola cliente '" + numeroClientes + "'.");
        socketCliente.close(); // cuando se ha finalizado de atender al
        // cliente se procede a cerrar el socket y
        // liberar los recursos asociados a éste
    } catch (Exception e){
        System.out.println(e.getMessage());
    }
}
System.out.println("Ya se han atendido a los 3 clientes en el puerto '" + pue
...
```

Implementación del cliente

```
...
try {
    Socket socketCliente = new Socket(host, puerto); // primitiva que
    // permite al cliente la conexión, en el host y puerto indicados,
    // con el servidor. Si no hubiera ningún proceso atendiendo
    // en esa máquina/puerto la petición se dispara una
    // excepción (indicada más abajo).
```

```

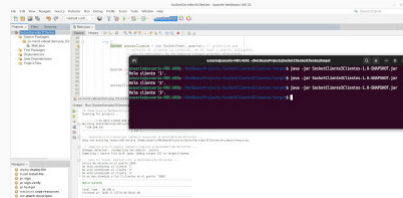
        System.out.println(          // salida por consola del mensaje
            (new DataInputStream(    // enviado desde el servidor
                socketCliente.getInputStream()).readUTF() );

        socketCliente.close(); // cuando se ha finalizado de realizar la
                               // solicitud del cliente se procede a cerrar
                               // el socket y liberar los recursos asociado
                               // a éste
    } catch (IOException e) {
        System.out.println(e.getMessage()); // se imprime por la salida
                                           // estándar el mensaje de error
    }
    ...

```

Hay varias formas de realizar la prueba:

- En una terminal de comandos se ejecuta el servidor y, en otra terminal, se ejecuta 3 veces el cliente
- En el entorno de desarrollo, en este caso NetBeans, se ejecuta el servidor y se observa el resultado en la pestaña **Output** mientras que, en una terminal de comandos se ejecuta 3 veces la versión del cliente.



Q Ministerio de Educación. (Uso educativo nc)

En ambos casos, para ejecutar en la consola de comandos, basta con invocar el archivo **.jar** de la siguiente manera: **java -jar Archivo.jar**

En la figura se muestra al servidor como ha procesado las solicitudes de tres clientes.



Código fuente completo del ejemplo

Aquí puedes descargar el código fuente completo:

 [Creación de Servidor que atiende 3 clientes empleando sockets](#) 
(PSP03_CONT_R016_SocketServidor3Clientes.zip)

 [Creación de Cliente que recibe saludo desde servidor empleando sockets](#)
(PSP03_CONT_R017_SocketCliente3Clientes.zip) 

3.- Sockets UDP



Caso práctico

Juan esta terminado de programar la aplicación.

María: —Hola **Juan**. Ya he recibido tu aplicación y va bastante bien. Para que puedas aprender más ahora vamos a ver la programación de sockets UDP. Como ya sabes, los sockets UDP se utilizan para cuando tenemos que enviar datos muy rápido y no nos preocupa que se pierda algún paquete.

Juan: —Estupendo María. Estoy deseando de empezar.



Q matylda (<https://www.flickr.com/photos/hackny/6202765137/>). (CC BY-SA)

En el caso de utilizar sockets UDP no se crea una conexión (como es el caso de socket TCP) y básicamente permite enviar y recibir mensajes a través de una dirección IP y un puerto. Estos mensajes se gestionan de forma individual y no se garantiza la recepción o envío del mensaje como si ocurre en TCP.

Para utilizar sockets UDP en java tenemos la clase `DatagramSocket` y para recibir o enviar los mensajes se utiliza clase `DatagramPacket`. Cuando se recibe o envía un paquete se hace con la siguiente información: mensaje, longitud del mensaje, equipo y puerto.



Para saber más

En los siguientes enlaces puedes ver los manuales oficiales de [HTML DatagramSocket](#) y [HTML DatagramPacket](#).

En el siguiente [enlace](#) podrás acceder a un artículo, que explica con un nivel de detalle elevado, las diferencias entre sockets haciendo uso del protocolo TCP respecto al protocolo UDP:

3.1.- Receptor

Los pasos que realiza la entidad receptora con el objeto de realizar la recepción de información en la comunicación son los siguientes:

- Publicar puerto: se utiliza el constructor de la clase `DatagramSocket` indicando el puerto por donde se van a recibir las conexiones.
- Esperar peticiones: en este momento el receptor, que tiene el rol de *servidor*, queda a la espera a que se conecte un emisor, que tiene el rol de *cliente* y está en disposición de recibir mensajes utilizando el método `receive` de la clase anterior.
- Procesar la información recibida (en este caso se muestra por la salida estándar).
- Cerrar/finalizar el uso de los recursos asociados a la comunicación.



Q Ministerio de Educación. (Uso educativo nc)

Es importante señalar que la comunicación, a este nivel, se realiza mediante *arrays* de *bytes*. Esto es así porque la trama, que es el paquete de datos asociado a un datagrama UDP, integra en este formato la información. Es por ello que, de forma previa a la recepción, es necesario declarar una variable de dicho tipo con un tamaño suficiente respecto al contenido de la comunicación:

```
byte[] cadena = new byte[1000];
```

Respecto a la construcción del datagrama, una vez creado el contenedor de la información en el paso anterior, solo queda invocar al constructor de la clase `DatagramPacket`:

```
DatagramPacket mensaje = new DatagramPacket(cadena,
cadena.length);
```

La recepción de la información (que conlleva decodificar desde el tipo `byte[]` a la clase `String`) y la subsiguiente liberación de recursos se logra con las sentencias siguientes:

```
datagramaSocket.receive(mensaje);
```

```
new String(mensaje.getData(),0,mensaje.getLength())
```

```
datagramaSocket.close();
```

Ejemplo

A continuación se muestra un extracto del código (con comentarios explicativos) de un programa que ejecuta el rol de servidor:

```

try {
    // primiti
    DatagramSocket datagramaSocket = new DatagramSocket(puerto); // p
    // al cliente la conexión, en el puerto indicado, con el
    // servidor. Si no hubiera ningun proceso atendiendo en esa
    // máquina/puerto la petición se dispara una excepción
    // (indicada más abajo).

    byte[] cadena = new byte[1000]; // en este array de bytes se alma
    // el mensaje contenido dentro de
    // datagrama UDP que será recibid

    // el datagrama será almacenado e
    // este objeto de la clase Datagr
    DatagramPacket mensaje = new DatagramPacket(cadena, cadena.length

    datagramaSocket.receive(mensaje); // escucha/recepción del datagr

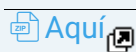
    System.out.println("Mensaje Recibido: " // esta la forma de conve
    // el array de bytes recibido en texto legible de
    // clase 'String':
    + new String(mensaje.getData(),0,mensaje.getLength()));

    datagramaSocket.close();// cuando se ha atendido la solicitud del
    // cliente se procede a cerrar el socket
    // liberar los recursos asociado a éste
} catch (IOException e) {
    System.out.println(e.getMessage()); // se imprime por la salida
    // estándar el mensaje de err
}

```



Código fuente del Receptor



Aquí puedes descargar el código fuente completo.

3.2.- Emisor

Por otro lado, para realizar una aplicación que implemente a la entidad emisora, con objeto de crear y realizar el envío de información en la comunicación, se deben seguir los siguientes pasos:



Q Ministerio de Educación. (Uso educativo [nc](#))

- Crear objeto que encapsule la conexión con el receptor: se utiliza el constructor de la clase `DatagramSocket` (en este caso, a diferencia del rol *receptor* que hemos visto anteriormente, no se especifica puerto).
- Crear objeto que contiene la información que se desea enviar al receptor: se crea, mediante el constructor con parámetros, un objeto de la clase `DatagramPacket` (es ahora cuando se especifica el mensaje a enviar, el equipo destinatario y el puerto que será empleado para establecer la comunicación).
- Enviar el paquete de datos: el método `send` del objeto creado antes realiza esta operación. Justo antes de la invocación se muestra en la salida estándar aquello que se quiere enviar así como los detalles (máquina destino y puerto).
- Cerrar/finalizar el uso de los recursos asociados a la comunicación.

Respecto a la construcción del datagrama, una vez creado el contenedor de la información en el paso anterior, solo queda invocar al constructor de la clase `DatagramPacket` con los parámetros específicos asociados al rol *emisor*:

```
DatagramPacket datagramaPaquete =  
DatagramPacket(mensaje, mensaje.length, InetAddress.getByName(host
```

En la anterior invocación se especifica, de izquierda a derecha, la información que se quiere enviar (en bytes) y su longitud, la dirección IP de la máquina de destino y el puerto de comunicaciones (deberá estar disponible ya que, en otro caso, devuelve error).

Ahora se invoca el método de envío, `send`, cuyo argumento es el objeto datagrama creado anteriormente. Tras ello se liberan los recursos solicitados (cierre de conexión con el método `close`):

```
datagramaSocket.send(datagramaPaquete);  
datagramaSocket.close();
```

Ejemplo

A continuación se muestra un extracto del código (con comentarios explicativos) de un programa que ejecuta el rol de cliente:

```

...
try {

                                                                    // primitiva
DatagramSocket datagramaSocket = new DatagramSocket(); // permite
// la conexión con el cliente. Si no hubiera ningún proceso cliente
// conectado esperará hasta que eso suceda

DatagramPacket datagramaPaquete // datagrama
    = new DatagramPacket(
        mensaje,           // información
        mensaje.length,    // longitud de la información
        InetAddress.getByName(host), // IP de máquina destino
        puerto);           // puerto de comunicación

System.out.println("Inicio de envío de información '" // aviso
                                                                    // salida estándar que info
        + new String(mensaje, StandardCharsets.UTF_8) // del mensa
        + "' en el puerto '" + puerto                  // del pue
        + "' a la máquina '" + host + "'"); // y de la máquina en
                                                                    // que se realizará la escu

datagramaSocket.send(datagramaPaquete); // envío del datagrama UDP
                                                                    // cliente

datagramaSocket.close(); // cuando se ha finalizado de atender
                                                                    // cliente se procede a cerrar el soc
                                                                    // y liberar los recursos asociados a
} catch (UnknownHostException e) { // error de host desconocido
    System.err.println("Desconocido: " + e.getMessage()); // inalcanz
} catch (SocketException e) { // error en socket
    System.err.println("Socket: " + e.getMessage());
} catch (IOException e) { // se imprime por la salida estándar el men
    System.out.println(e.getMessage()); // de error
}

...

```



Código fuente del Emisor

Aquí puedes descargar el código fuente completo:

 [Creación de emisor empleando sockets UDP](#) 



Autoevaluación

Indica la característica que no pertenece a los sockets UDP.

- ☐ Utiliza un determinado puerto.
- ☐ Establece una conexión entre un cliente/servidor.
- ☐ Permiten enviar y recibir paquetes.
- ☐ En cada paquete va la dirección y puerto destino.

Mostrar retroalimentación

Solución

1. Incorrecto (#answer-36_58)
2. Correcto (#answer-36_128)
3. Incorrecto (#answer-36_130)
4. Incorrecto (#answer-36_132)

3.3.- Ejemplo

Para continuar con el ejemplo anterior, y aprender a utilizar los sockets UDP, se muestra un ejemplo sencillo donde intervienen dos procesos:



Q Samir P. (CC BY-SA)

- **ReceptorUDP:** inicia la escucha/recepción de datagramas UDP en el puerto 2000 y muestra en pantalla todos los mensajes que llegan a él (funciona de forma continuada hasta que lo detengamos manualmente mediante un bucle infinito).
- **EmisorUDP:** envía, mediante invocación con argumentos y ejecución por línea de comandos, mensajes al receptor/destinatario a través del puerto 2000.

Debido a que el código fuente del emisor es idéntico al suministrado en apartados anteriores ahora solo se extraen las partes diferentes del receptor (servidor) para que haga *escucha continua* (bucle `while(true)`):

```
...
DatagramSocket datagramaSocket = null;

try {
    ...
    System.out.println("Proceso servidor/receptor a la espera de "
        + "comunicaciones enviadas por clientes/emisores ...");

    while(true){
        datagramaSocket.receive(mensaje);    // escucha y/o recepción del
                                              // datagrama

        ...
    }
} catch (IOException e) {
    ...
} finally {
    if(datagramaSocket!=null)
        datagramaSocket.close();// cuando se ha atendido la solicitud del
                                // cliente se procede a cerrar el socket y
                                // liberar los recursos asociado a éste
}
...
```

Para realizar las pruebas se opera de forma similar a la indicada en el ejemplo de las aplicaciones del lado cliente y del lado servidor para el protocolo TCP.





Aquí puedes descargar el código fuente del receptor UDP que realiza una escucha continua hasta que es detenido manualmente:

 [Creación de Receptor UDP que atiende emisores ininterrumpidamente empleando sockets \(PSP03_CONT_R024_ReceptorUDPejemplo.zip\)](#) 